

ROBO 5302 (CSCI 4302/5302) - Advanced Robotics
Homework #3 (Assigned: Fri 9/26, Due: Fri 10/24 11:59pm on Canvas)
Markov Decision Processes

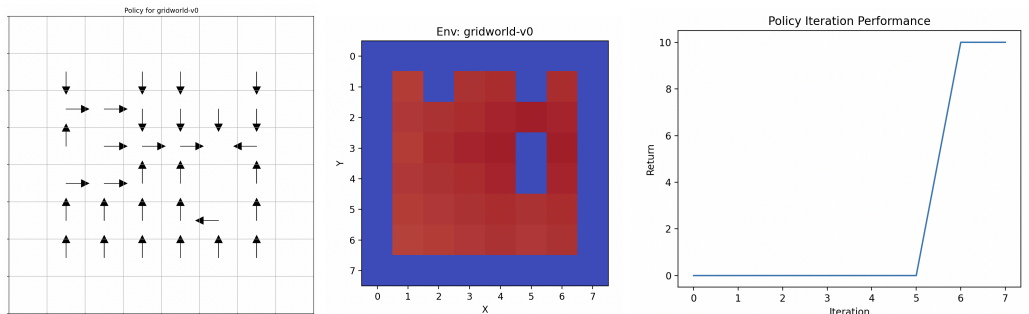
Isaac Edelman

GitHub: https://github.com/isaedelman/ROBO_5302.git

1) Policy Iteration [30 points]

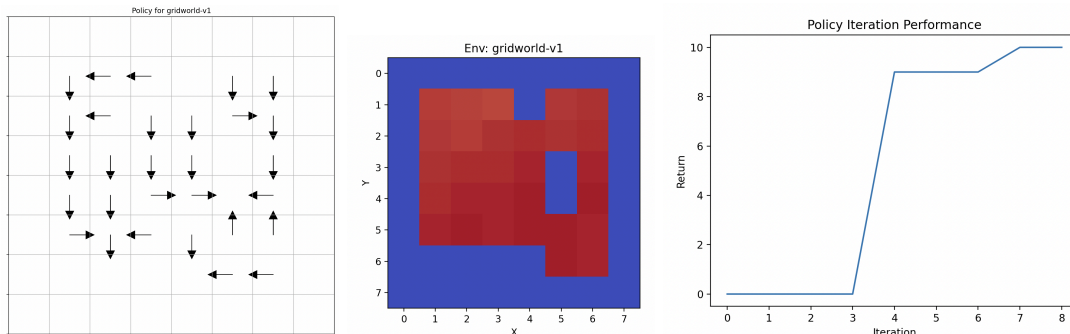
(Environment: plan0.txt):

Policy Plot, Value Plot, Return as a function of iteration:



(Environment: plan1.txt):

Policy Plot, Value Plot, Return as a function of iteration:



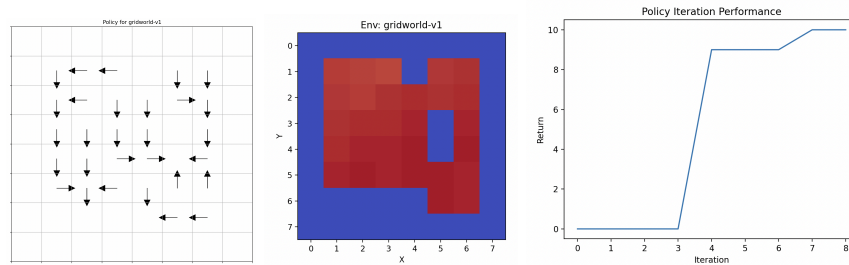
Written Question:

1. Try a few different values of γ for your policy iteration algorithm ($\gamma = 0.5, 0.75, 0.9$ and 0.99 may be good choices).

Does your policy change based on the value of γ ? Why is this the case? Why not?

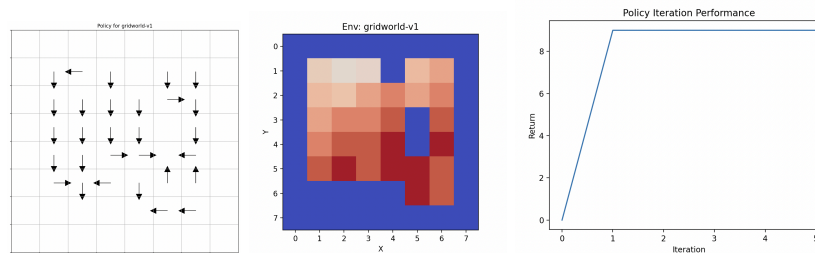
(Environment: plan1.txt) $\gamma=0.99$:

Policy Plot, value plot, Return as a function of iteration:



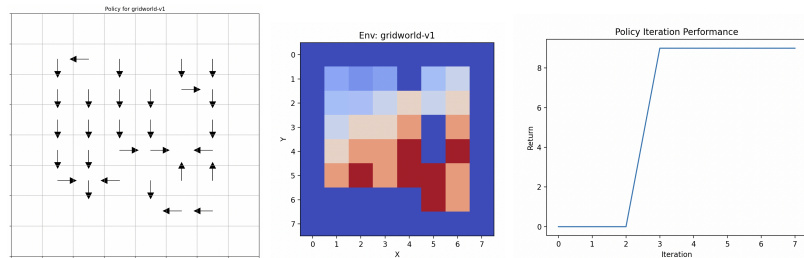
(Environment: plan1.txt) $\gamma=0.90$:

Policy Plot, value plot, Return as a function of iteration:



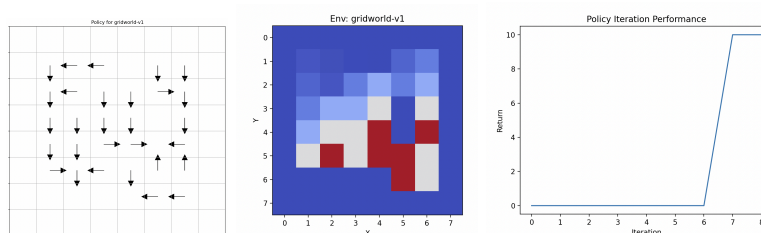
(Environment: plan1.txt) $\gamma=0.75$:

Policy Plot, value plot, Return as a function of iteration:



(Environment: plan1.txt) $\gamma=0.50$:

Policy Plot, value plot, Return as a function of iteration:

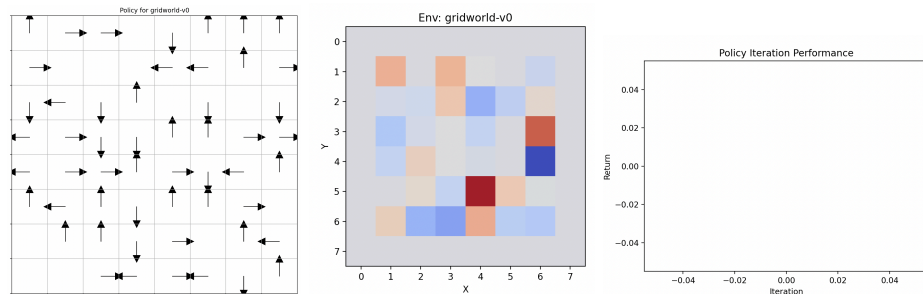


The policy does not change; however, if γ is very low, then the policy might not be able to converge because of limited steps. The value function and, in turn, the plot will change because the value of each step is changing due to γ valuing long-term/short-term results differently.

- 2) **Bonus Question [10 points]:** This problem can also be solved exactly (instead of iteratively) by casting the unknown state values as a set of linear equations. Implement this solution method and show in your writeup that you get the same results as for your policy iteration above. Also comment on the runtime between the exact and iterative approach.

(Environment: plan0.txt):

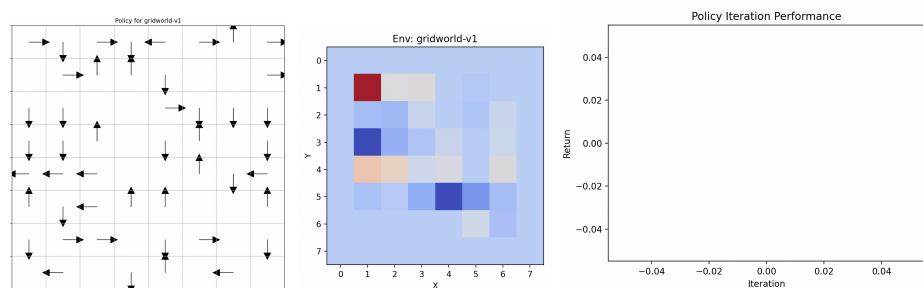
Policy Plot, Value plot, Return as a function of iteration:



Runtime comparison: Exact policy done in 0.0049 seconds
Computed Q1 Deterministic_PI Policy in 0.0379989 seconds

(Environment: plan1.txt):

Policy Plot, Value plot, Return as a function of iteration:



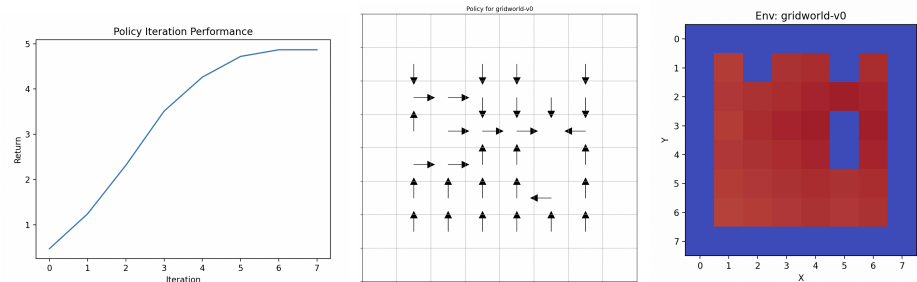
Runtime comparison: Exact policy done in 0.0116 seconds
Computed Q1 Deterministic_PI Policy in 0.050416 seconds

For small gridworlds, the exact method will usually be faster due to the matrix form. However, for large-scale RL problems around a thousand states the matrix inversion becomes computationally expensive and slower due to the computational complexity exploding. I think for space complexity the exact solution is $O(n^2)$ and it will have a time complexity of $O(n^3)$ while the iterative method will have a space complexity of $O(k n^2)$ and a time complexity of $O(n)$.

3) Value Iteration [20 points]

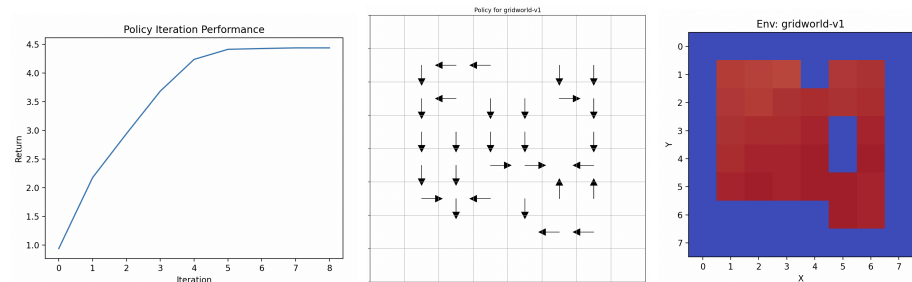
(Environment: plan0.txt):

Policy Plot, Value Plot, Return as a function of iteration:



(Environment: plan1.txt):

Policy Plot, Value Plot, Return as a function of iteration:



Written Questions:

1. Are your policy and value iteration results the same? Do you expect them to be?

Yes, the policy and value iteration results are the same. I expect them to be the same as long as they are both allowed to converge fully. They are both just finding the best policy, but in different ways. Policy iteration alternates between policy evaluation and policy improvement, while value iteration combines the policy evaluation and policy improvement into one step.

2. Compare the runtime of your policy and value iteration implementations. Which runs faster? Why?

plan0

Computed Q2 Deterministic_PI Policy in 0.0368741 seconds

plan1

Computed Q1 Deterministic_PI Policy in 0.045897 seconds

plan0

Computed Q2 Deterministic_VI Policy in 0.026119 seconds

plan1

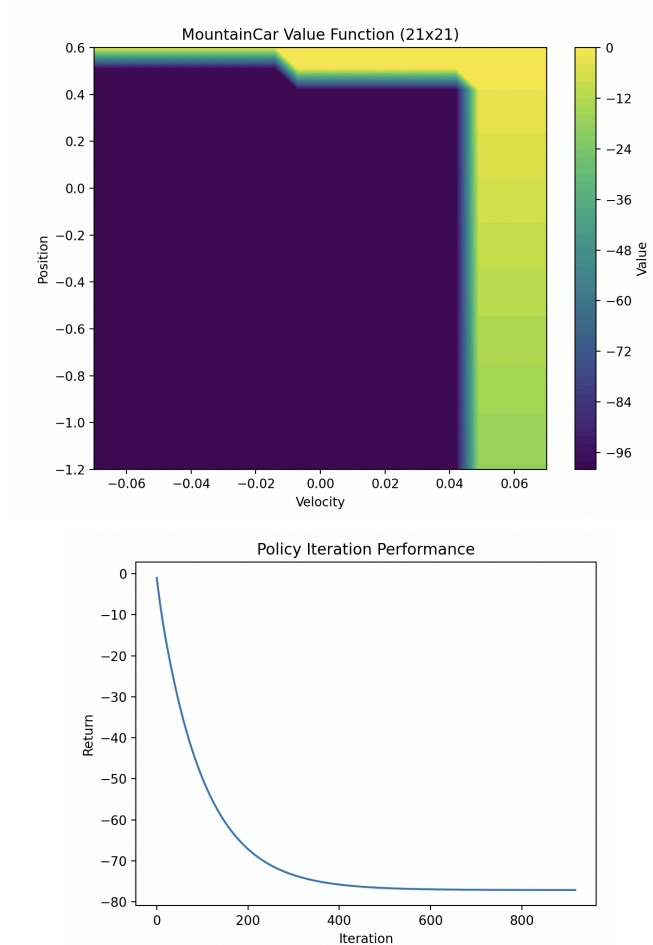
Computed Q2 Deterministic_VI Policy in 0.0373838 seconds

I think the performance difference occurs because of a couple of reasons. First, the value iteration updates state values directly in each iteration without solving a separate policy evaluation step to convergence, making it computationally lighter per iteration. Policy iteration, on the other hand, alternates between evaluating the current policy (which can be costly) and improving it, leading to fewer total iterations but higher per-iteration computation. For small problems, both methods are efficient, but value iteration typically runs faster due to its simpler update structure.

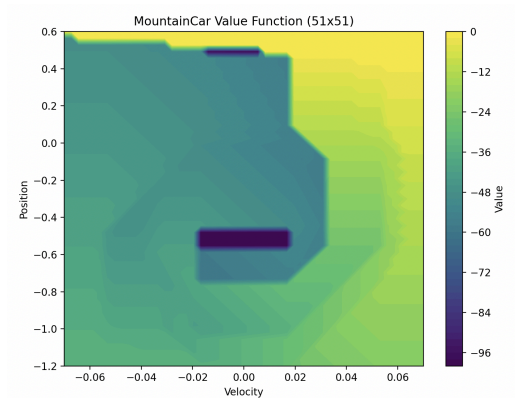
4) Nearest neighbors interpolation [20 points]

Discretization resolution: gamma=0.99, tol=1e-4

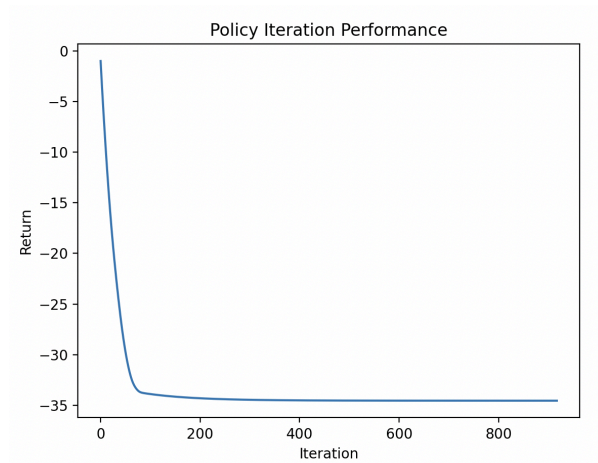
[21]:



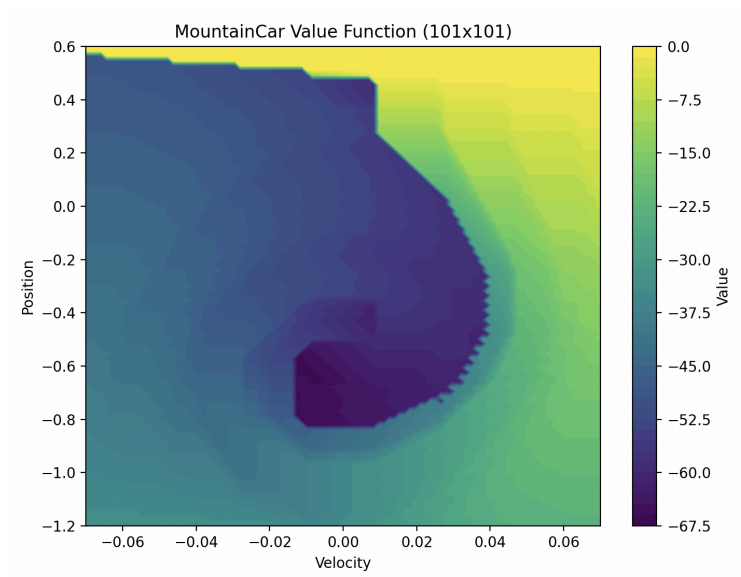
[51]:



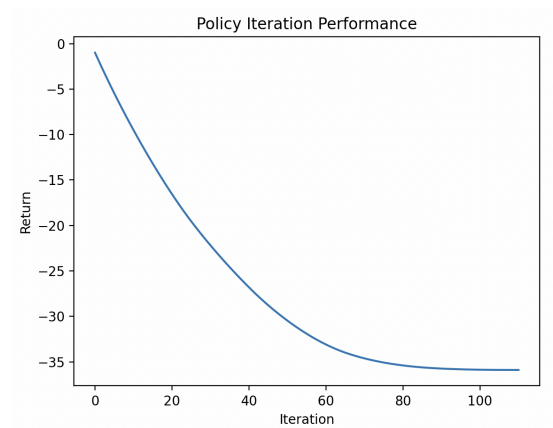
Last policy reward:



[101]:



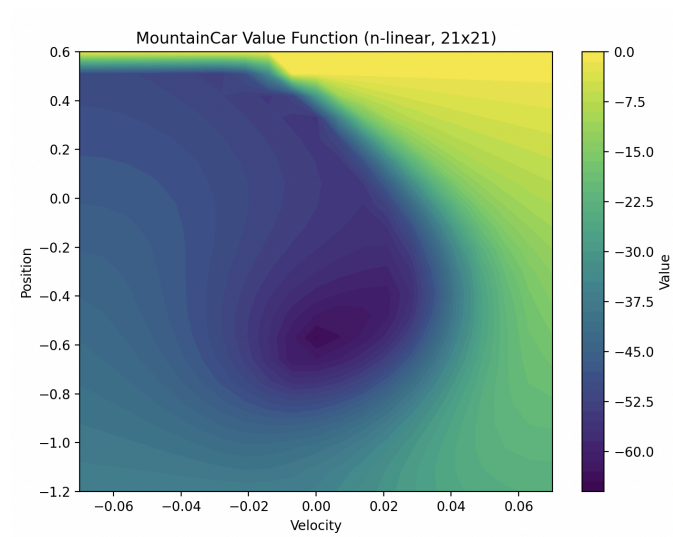
Last policy reward:



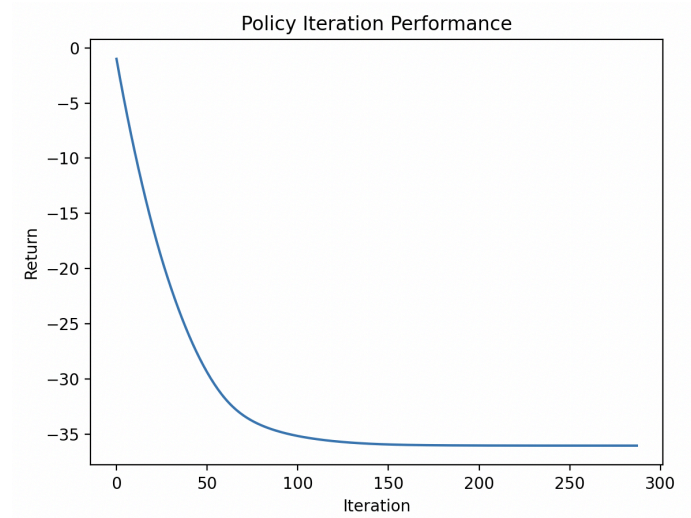
5) Linear Interpolation [30 points] [GRAD QUESTION]

Discretization resolution: $\gamma=0.99$, $\text{tol}=1e-4$

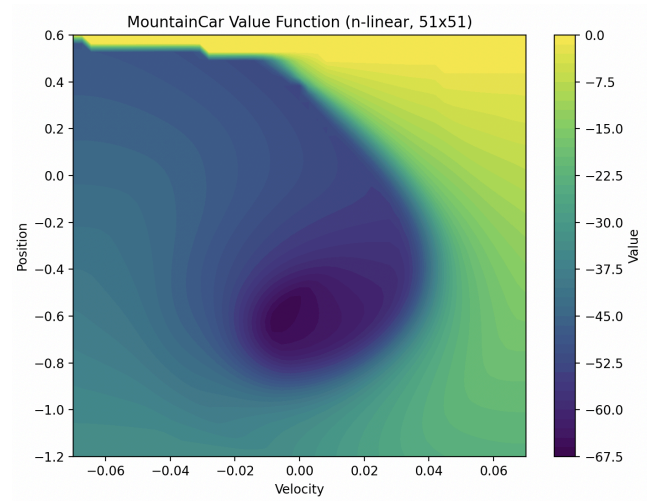
[21]:



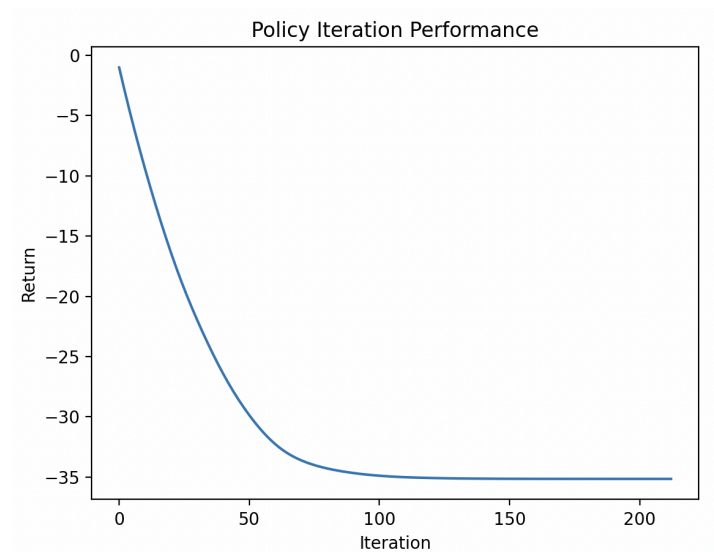
Last policy reward:



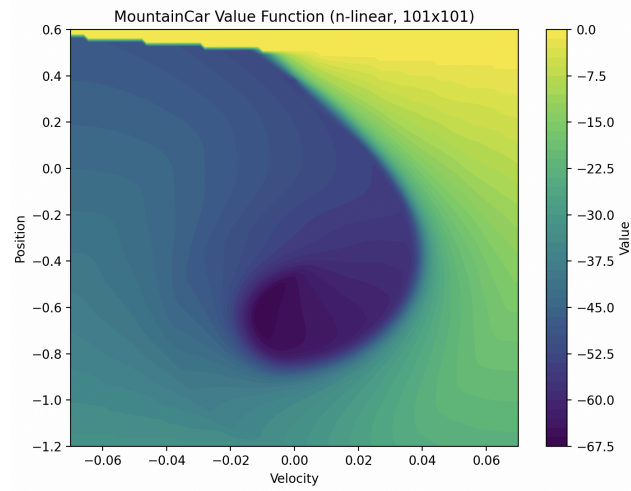
[51]:



Last policy reward:



[101]:



Last policy reward:

